Building a RAG-powered Chatbot

End-to-End: Ingest, Embed, Retrieve, Generate

Day 3 - Session 3

KavinSchool

Session 3: What We'll Build

An **end-to-end RAG chatbot** that:

1. Ingests a small knowledge base (markdown docs)
2. Chunks and embeds the documents
3. Stores embeddings in a local FAISS index
4. Answers user questions grounded in the knowledge base
5. Cites sources in every response

Step 1: Ingestion Pipeline

Loading and Chunking Documents

KavinSchool

Loading Documents

```
import os

def load_documents(docs_dir):
    """Load all .md files from a directory."""
    documents = []
    for filename in sorted(os.listdir(docs_dir)):
        if filename.endswith(".md"):
            filepath = os.path.join(docs_dir, filename)
            with open(filepath, "r") as f:
                documents.append({
                    "filename": filename,
                    "content": f.read(),
                })
    return documents

docs = load_documents("day3/knowledge-base")
print(f"Loaded {len(docs)} documents")
```

Chunking with Metadata

```
def chunk_documents(documents, chunk_size=500, overlap=50):
    """Chunk documents and preserve source metadata."""
    chunks = []
    for doc in documents:
        text = doc["content"]
        start = 0
        chunk_num = 0
        while start < len(text):
            end = start + chunk_size
            chunk_text = text[start:end]
            chunks.append({
                "text": chunk_text,
                "source": doc["filename"],
                "chunk_id": chunk_num,
            })
            start = end - overlap
            chunk_num += 1
    return chunks

chunks = chunk_documents(docs)
print(f"Created {len(chunks)} chunks")
```

Step 2: Embedding & Storage

From Text Chunks to Vectors

KavinSchool

Embedding All Chunks

```
import litellm
import numpy as np

def embed_texts(texts, model="text-embedding-3-small"):
    """Embed a list of texts using litellm."""
    response = litellm.embedding(model=model, input=texts)
    return [item["embedding"] for item in response.data]

# Embed all chunks (batch for efficiency)
texts = [chunk["text"] for chunk in chunks]
embeddings = embed_texts(texts)
print(f"Generated {len(embeddings)} embeddings, "
      f"dimension: {len(embeddings[0])}")
```

Cost check: 50 chunks x ~100 tokens each = 5,000 tokens

At $0.02 per 1M tokens = **$0.0001** (essentially free for prototyping)

Storing in FAISS

```
import faiss

def build_index(embeddings):
    """Build a FAISS index from a list of embeddings."""
    dimension = len(embeddings[0])
    index = faiss.IndexFlatL2(dimension)

    vectors = np.array(embeddings, dtype=np.float32)
    index.add(vectors)

    return index

index = build_index(embeddings)
print(f"FAISS index contains {index.ntotal} vectors")
```

`IndexFlatL2` does exact (brute-force) search. Fine for < 100K vectors. For millions, use `IndexIVFFlat`.

Saving and Loading the Index

```
# Save — so you don't re-embed every time
faiss.write_index(index, "knowledge_base.faiss")

# Load
index = faiss.read_index("knowledge_base.faiss")
```

> In production, you'd also persist the chunks list (e.g., as JSON or SQLite) alongside the FAISS index.

Step 3: Query Pipeline

Retrieve → Augment → Generate

Retrieving Relevant Chunks

```
def retrieve(query, index, chunks, k=3):
    """Embed the query, search the index, return top-k chunks."""
    query_embedding = embed_texts([query])[0]
    query_vector = np.array([query_embedding], dtype=np.float32)

    distances, indices = index.search(query_vector, k)

    results = []
    for i, idx in enumerate(indices[0]):
        results.append({
            **chunks[idx],
            "score": float(distances[0][i]),
        })
    return results
```

Augmenting the Prompt with Context

```
def build_rag_prompt(query, retrieved_chunks):
    """Build a prompt with retrieved context."""
    context = "\n\n---\n\n".join(
        f"[Source: {c['source']}]\n{c['text']}"
        for c in retrieved_chunks
    )

    messages = [
        {"role": "system", "content":
            "Answer the user's question based ONLY on the provided context. "
            "If the context doesn't contain the answer, say 'I don't have enough information to answer that.' "
            "Always cite your sources using [Source: filename]."},
        {"role": "user", "content":
            f"Context:\n{context}\n\n---\n\nQuestion: {query}"},
    ]
    return messages
```

> The key instruction: **"based ONLY on the provided context."** This is what prevents hallucination.

The Complete RAG Query

```
def rag_query(query, index, chunks, k=3):
    """Full RAG pipeline: retrieve → augment → generate."""
    # Step 1: Retrieve
    retrieved = retrieve(query, index, chunks, k=k)

    # Step 2: Augment prompt
    messages = build_rag_prompt(query, retrieved)

    # Step 3: Generate
    response = get_completion(messages, temperature=0.0)

    return {
        "answer": response,
        "sources": [c["source"] for c in retrieved],
        "chunks_used": len(retrieved),
    }

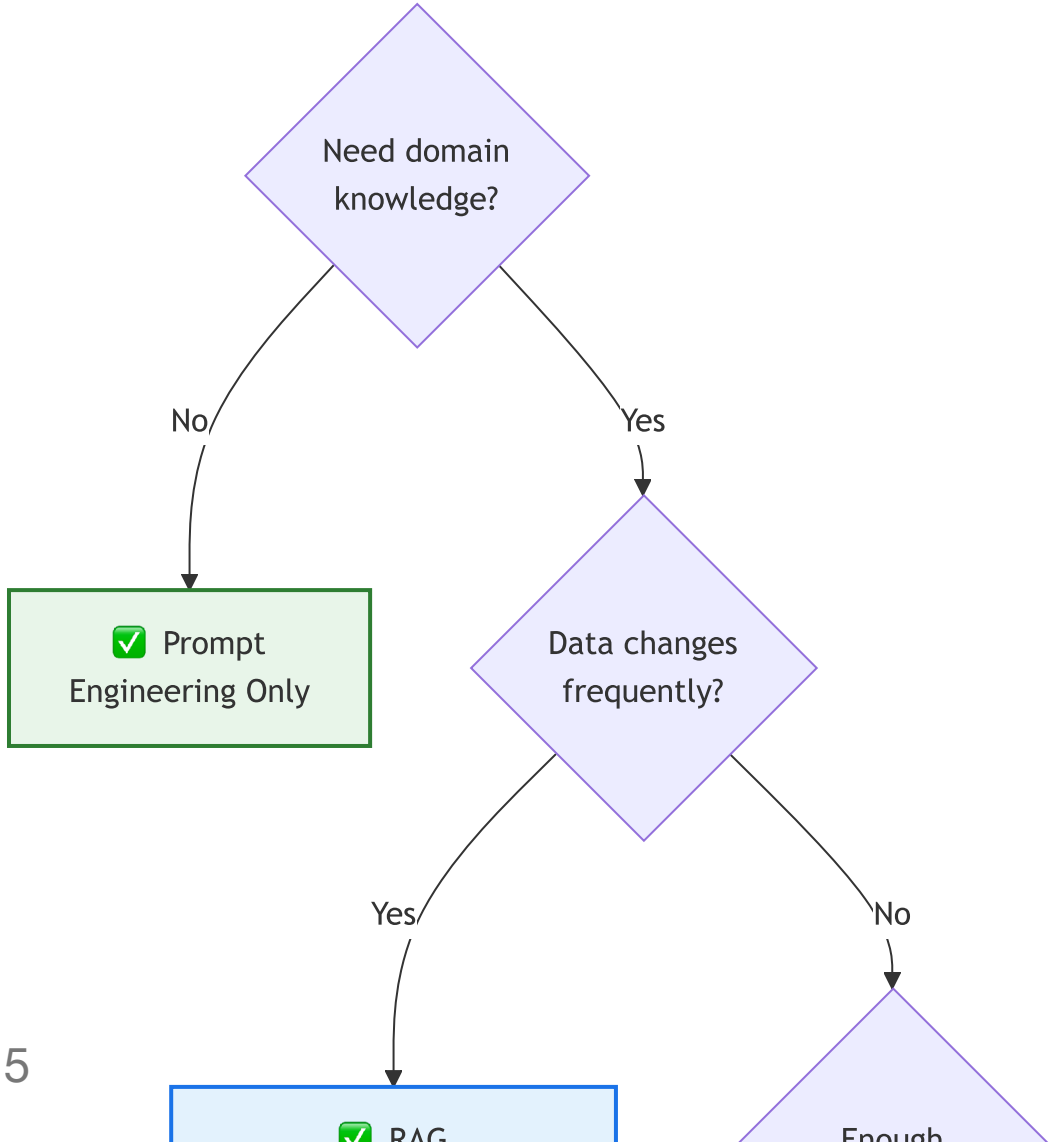
result = rag_query("What is the vacation policy?", index, chunks)
print(result["answer"])
print(f"Sources: {result['sources']}")
```

RAG vs Alternatives

When Is RAG the Right Choice?

KavinSchool

RAG vs Fine-Tuning vs Prompt Engineering



Decision Guide

Approach	When to Use	Cost	Effort
Prompt Engineering	General tasks, no private data	Low	Low
RAG	Private/changing data, need citations	Medium	Medium
Fine-Tuning	Specific style/behavior, stable data	High	High
RAG + Fine-Tuning	Enterprise production systems	Highest	Highest

Rule of thumb: Try prompt engineering first. Add RAG when you need private data. Fine-tune only as a last resort.

Common RAG Pitfalls

Pitfall	Symptom	Fix
Chunks too large	Irrelevant context dilutes the answer	Reduce chunk size to 200-300 tokens
Chunks too small	Missing important context	Increase chunk size, add overlap
Bad embedding model	Poor retrieval quality	Try a larger embedding model
Too few results (low k)	Misses relevant info	Increase k from 3 to 5
Too many results (high k)	Noise overwhelms signal	Decrease k, add score threshold
No source citation	Users can't verify answers	Add citation instructions to prompt

Session 3: Key Takeaways

1. **RAG has clear steps.** Load → chunk → embed → store → retrieve → generate
2. **Metadata is essential.** Track source file and chunk position for citations
3. **FAISS is great for prototyping.** Local, fast, no server needed
4. **The prompt does the grounding.** "Answer ONLY from the provided context"
5. **Try RAG before fine-tuning.** It's cheaper, faster, and data stays fresh

> Next: Session 4, we wrap the chatbot in a Flask API!

Questions?

Session 3 Complete. Break Time!

KavinSchool